

Ticker Metrics Terminal

Project Documentation Report

An interactive equity metrics, comparison, and risk-classification tool
built with Python, Streamlit, Plotly, and pandas

1. Project Overview

The Ticker Metrics Terminal is a data-driven financial analysis application built entirely in Python. It was developed to solve a specific problem: given a raw Excel dataset containing market metrics for 1,591 equity tickers, provide a fast, visual, and non-technical way for a user to look up any individual ticker, compare multiple tickers side-by-side, and understand each ticker's relative risk profile - including a data-driven suggested leverage figure for each equity.

The project evolved through several stages: a desktop application built with Python's built-in Tkinter toolkit, followed by a full migration to a browser-based Streamlit web application with interactive Plotly charts and a custom dark 'terminal' visual theme. Both versions read from the same underlying dataset and share the same core data-processing and risk-scoring logic.

1.1 Source Data

The application reads from a single Excel workbook, **Ticker_Data.xlsx**, specifically the 'Ticker Metrics' sheet, which contains one row per ticker with the following columns:

Column	Description
Ticker	The ticker symbol and exchange, e.g. "NVDA US Equity"
Volume	Trading volume (shares)
Volume in USD	Trading volume converted to US Dollars
Market Cap	Market capitalization in US Dollars
Historical Volatility 30D / 60D / 90D	Annualized historical volatility over 30/60/90-day windows
Beta	The ticker's beta (systematic risk relative to the market)

The dataset contains **1,591 unique tickers** spanning global equities across multiple exchanges.

2. Objectives

- Allow a user to search for any ticker - by symbol *or* by common company name - and instantly view its key market metrics.
- Provide a side-by-side comparison tool for up to 10 tickers across any single metric, plus a normalized multi-metric "risk profile shape" view.
- Automatically classify every ticker into a risk tier (Very Stable to Very Risky) using a transparent, weighted scoring methodology, and translate that into a suggested leverage/margin figure per ticker.
- Present all of the above through a polished, interactive, and easy-to-navigate interface suitable for someone with no technical background.
- Make the whole project deployable and shareable: version-controlled on GitHub and runnable either as a local desktop app or as a Streamlit web app.

3. Technology Stack

Layer	Technology	Purpose
Language	Python 3.13	Core application language
Data handling	pandas, openpyxl	Reading and processing the Excel dataset
Desktop UI (v1)	Tkinter (built-in)	Original local desktop application
Desktop charts (v1)	Matplotlib	Static charts embedded in the Tkinter UI
Web UI (v2)	Streamlit	Browser-based interactive web application
Web charts (v2)	Plotly	Interactive, hoverable charts in the web app
Version control	Git & GitHub	Source control and project hosting

4. Project Structure

The codebase is organized into small, single-purpose modules rather than one large file, so each piece of logic is easy to locate, test, and modify independently.

```
TickerApp/  
■■■ app.py # Streamlit web app (main entry point, v2)  
■■■ main.py # Tkinter desktop app entry point (v1)  
■■■ ui.py # Tkinter desktop UI (v1)  
■■■ charts.py # Matplotlib chart builders (v1, desktop)  
■■■ charts_plotly.py # Plotly interactive chart builders (v2, web)  
■■■ data_loader.py # Excel loading, search, and lookup logic  
■■■ risk_classifier.py # Risk scoring, grouping, and leverage engine  
■■■ company_names.py # Company name to ticker symbol mapping  
■■■ formatters.py # Number formatting helpers ($5.42T, etc.)  
■■■ requirements.txt # Python package dependencies  
■■■ Ticker_Data.xlsx # Source dataset
```

4.1 Module Responsibilities

Module	Responsibility
data_loader.py	Loads the Excel file once at startup, provides search-by-symbol and search-by-name, popular-ticker suggestions, per-ticker metric lookup, market-wide averages, and risk-group browsing.
risk_classifier.py	Computes the weighted Risk Score, assigns each ticker to one of 5 risk tiers, and calculates a Suggested Leverage value.
company_names.py	Maps ~60 common company names (e.g. "Apple", "Tesla") to their ticker root symbol, enabling name-based search.
formatters.py	Converts raw numeric values into readable display strings (e.g. 5,419,832,000,000 -> "\$5.42T").
charts_plotly.py / charts.py	Build every chart shown in the app (volatility bars, market comparisons, radar charts, risk-group summaries), styled to match the app theme.
app.py / ui.py	Assemble the actual screens, tabs, and interactive widgets, wiring the data and chart modules together.

5. Core Features

5.1 Ticker Lookup

The Lookup screen lets a user find any of the 1,591 tickers either by typing its exact symbol (e.g. "AAPL") or a common company name (e.g. "Apple"). A curated set of 12 well-known "Popular Tickers" (Nvidia, Apple, Microsoft, Google, Amazon, Meta, Tesla, AMD, JPMorgan, Johnson & Johnson, ExxonMobil, and Visa) is shown as one-click buttons before any search is performed, so a first-time user is never facing an empty search box with no starting point.

Once a ticker is selected, the app displays:

- Market Cap, Volume, Volume in USD, Beta, and the assigned Risk Group and Suggested Leverage as metric cards.
- A bar chart of 30/60/90-day historical volatility.
- A grouped bar chart comparing the selected ticker's Beta and 30-day Volatility against the dataset-wide average.
- A gauge chart showing the ticker's Market Cap percentile rank relative to all other tickers in the dataset.

5.2 Compare Tickers

The Compare screen allows a user to select between 2 and 10 tickers (searchable by symbol or company name, with the same Popular Tickers quick-add shortcuts) and compare them on any single chosen metric - Market Cap, Volume, Beta, Risk Score, or any of the three volatility windows.

- A grouped bar chart of the selected metric across all chosen tickers.
- A radar (spider) chart showing each ticker's normalized "risk profile shape" across five dimensions: Beta, Risk Score, and Volatility 30D/60D/90D - useful for visually spotting which tickers share a similar risk fingerprint.
- A full comparison data table listing every metric for every selected ticker side by side.

5.3 Risk Groups

This is the most analytically significant feature of the project: an automated system that classifies every one of the 1,591 tickers into one of five risk tiers, and assigns each a suggested leverage figure. The full methodology is detailed in Section 6.

- An overview chart showing how many tickers fall into each risk tier.
- An overview chart showing the average suggested leverage per tier.
- A browsable, scrollable table showing every ticker within a chosen tier (not a limited sample), ordered so the most representative extreme of that tier appears first.

6. Risk Classification Methodology

A core requirement of this project was to group all equities into stable vs. risky tiers, using a specific priority order of factors: **(1) Market Cap, (2) Volume in USD, (3) Beta, (4) 60-Day Volatility**. The methodology below implements that priority order as a transparent, reproducible weighted scoring formula, rather than an opaque black-box model.

6.1 Step 1 - Percentile Ranking

Each of the four metrics is converted into a percentile rank from 0.0 to 1.0 across all 1,591 tickers. A percentile of 0.95 for Market Cap means the ticker's market capitalization is larger than 95% of all other tickers in the dataset. This step is essential because the four metrics exist on wildly different numeric scales (Market Cap in trillions vs. Beta typically between 0 and 2) and cannot be combined directly without first normalizing them onto a common 0-1 scale.

6.2 Step 2 - Weighted Composite Risk Score

The four percentile ranks are combined into a single Risk Score using weights that directly mirror the requested priority order:

```
Risk Score = 0.40 x (1 - Market Cap percentile) + 0.30 x (1 -  
Volume in USD percentile) + 0.20 x (Beta percentile)  
+ 0.10 x (Volatility 60D percentile)
```

Market Cap and Volume in USD are **inverted** (1 minus the percentile) because a larger, more liquid company is inherently more stable - so a high Market Cap percentile should *lower* the risk score, not raise it. Beta and Volatility are used directly, since a higher value in either genuinely indicates higher risk. The resulting Risk Score ranges from 0.0 (safest possible ticker) to 1.0 (riskiest possible ticker).

6.3 Step 3 - Risk Tier Assignment

Rather than forcing an artificial equal split (e.g. exactly 20% of tickers in each of 5 tiers), the actual distribution of Risk Scores across the dataset was studied first. It was found to be roughly bell-shaped, ranging from 0.067 to 0.989, with no sharp natural breakpoints. Fixed, evenly-spaced thresholds were therefore chosen instead, so that each tier's size reflects how tickers genuinely distribute across the risk spectrum:

Risk Score Range	Tier	Tickers in Dataset
0.00 - 0.20	Very Stable	84
0.20 - 0.40	Stable	420
0.40 - 0.60	Moderate	564
0.60 - 0.80	Risky	452
0.80 - 1.00	Very Risky	71

This produced a realistic, data-driven spread rather than an artificial one - for example, the "Moderate" tier is the largest because most tickers in this dataset genuinely score near the middle of the risk spectrum, while very few score at either extreme.

6.4 Step 4 - Suggested Leverage

The Risk Score is converted into a Suggested Leverage figure using a straight-line (linear) formula between a safest-case maximum leverage and a riskiest-case minimum leverage:

$$\text{Suggested Leverage} = 10.0 - (\text{Risk Score} \times (10.0 - 1.0))$$

A Risk Score of 0.0 (safest) yields a suggested leverage of 10.0x. A Risk Score of 1.0 (riskiest) yields 1.0x. This directly encodes the requested business rule: safer, larger, more liquid stocks with lower Beta and Volatility receive a higher suggested leverage ceiling, while riskier, smaller, less liquid, more volatile stocks receive a lower one.

Risk Tier	Avg. Suggested Leverage	Avg. Market Cap	Avg. Beta
Very Stable	8.70x	\$349.2B	0.53
Stable	7.23x	\$173.2B	0.84
Moderate	5.48x	\$24.3B	0.87
Risky	3.82x	\$6.5B	0.96
Very Risky	2.36x	\$2.2B	1.21

Figures computed from the actual Ticker_Data.xlsx dataset (1,591 tickers).

Important note: This is a data-driven analytical heuristic intended for internal decision support and portfolio analysis, not a certified financial risk model or licensed investment advice. The suggested leverage figures should be treated as a starting point for discussion with a risk or compliance function, not as a final, standalone recommendation. Both the weights and the leverage range are configurable in a single location (risk_classifier.py) so the methodology can be adjusted as institutional policy requires.

7. User Interface Evolution

7.1 Version 1 - Tkinter Desktop Application

The first version was built using Tkinter, Python's built-in GUI toolkit, chosen specifically because it requires no external accounts, no sign-ins, and no internet connection to run - only Python itself. It featured a dark, professional color theme, a tabbed interface (Lookup / Compare / Risk Groups), embedded Matplotlib charts, and custom-built scrollable areas for long result lists.

7.2 Version 2 - Streamlit Web Application

The project was subsequently rebuilt as a Streamlit web application to enable browser-based access, more sophisticated interactivity, and easier deployment/sharing. Key upgrades in this version include:

- Fully interactive Plotly charts (hover tooltips, zoom, pan) replacing the static Matplotlib images.
- Native searchable dropdowns and multi-select widgets, replacing custom-built autocomplete logic.
- Native scrolling data tables (no custom scrollbar code required), allowing the Risk Groups tab to display every ticker in a selected tier rather than a limited sample.
- A custom visual theme: a pure black background with a subtle radial gradient and faint grid overlay, glass morphism (semi-transparent, blurred) metric cards with layered depth shadows, a highlighter-yellow title, and neon cyan/green/pink accents throughout.
- Session-state-driven navigation, including "Back" controls on both the Lookup and Compare screens.

7.3 Version 3 - Company Name Search

Because the source dataset only contains ticker symbols (e.g. "NVDA US Equity") and not full company names, a dedicated mapping module (`company_names.py`) was built covering roughly 60 well-known companies. This allows a user unfamiliar with ticker symbols to search naturally by typing "Apple", "Tesla", or "Nvidia" and still be routed to the correct ticker.

8. Deployment and Version Control

The complete project - all Python modules, the requirements file, and the source Excel dataset - is version-controlled using Git and hosted on GitHub, enabling change tracking and straightforward collaboration or handover.

- Local development and testing were performed via VS Code, using an Anaconda Python distribution.
- Dependencies (pandas, openpyxl, streamlit, plotly) are declared in `requirements.txt` and installed via pip.

- The Streamlit application (app.py) can be run locally with a single command: streamlit run app.py, which launches a local web server and opens the app in the default browser.
- The application is additionally structured to be deployable to Streamlit Community Cloud directly from the GitHub repository, for browser-based access without any local installation.

9. Potential Future Enhancements

- Incorporate live/real-time market data via an external API, rather than a static Excel snapshot.
- Add historical price trend charts using the per-ticker "History" sheets already present in the source workbook.
- Allow the risk-scoring weights and leverage range to be adjusted directly within the app UI rather than only in source code.
- Add user authentication and saved watchlists for a multi-user deployment.
- Export comparison results and risk-group tables to Excel or PDF directly from the app.

10. Conclusion

The Ticker Metrics Terminal successfully transforms a static, 1,591-row Excel spreadsheet into an interactive analytical tool covering individual ticker lookup, multi-ticker comparison, and a transparent, weighted risk classification and leverage suggestion system. The project demonstrates a complete workflow from raw data ingestion through to a polished, interactive, deployable web application, with all core logic - data loading, search, risk scoring, and formatting - cleanly separated from the presentation layer so that either UI (desktop or web) can be maintained independently on top of the same underlying analytical engine.

MADE BY NIHAL LOPEZ